Compte Rendu TP2

Ahmed Riadh Hedhili

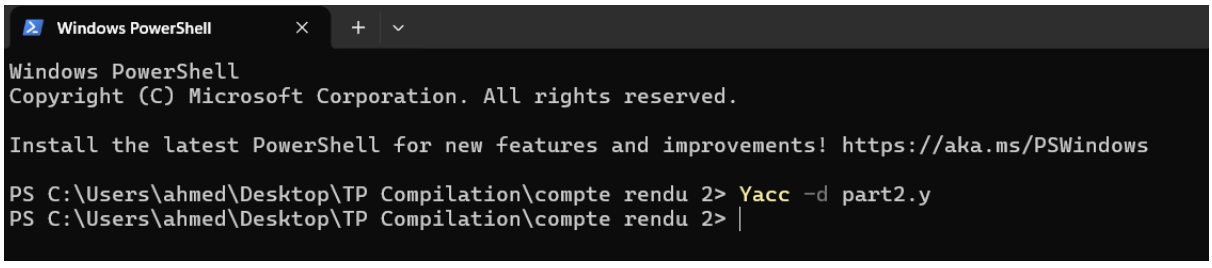
L2CS02

Part 2:

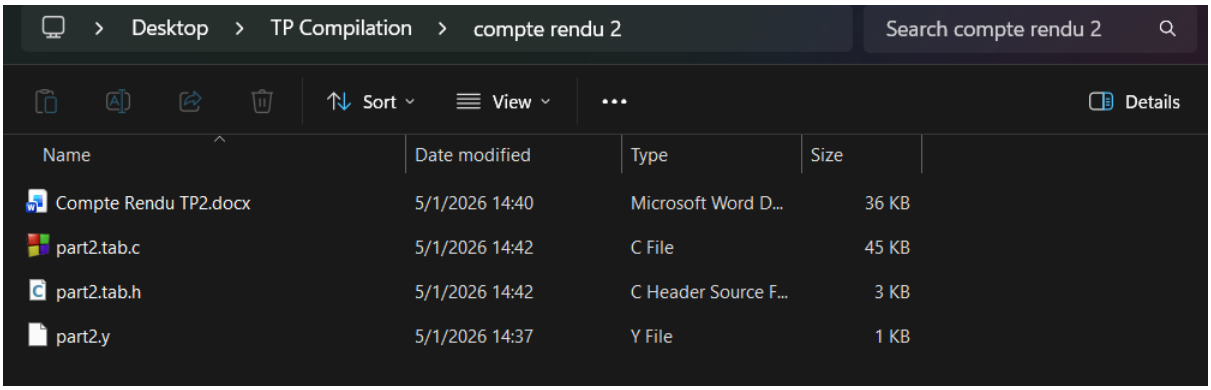
1)

```
%{
#include <stdio.h>
int yylex(void);
int yyerror (char*);
%}
%%
mot : S '$' {printf("mot correct\n"); getchar();}
;
S : 'a'S'a'
| 'b'S'b'
| 'c'
;
%%
int yylex()
{
char c=getchar();
if (c=='a'||c=='b'||c=='c'||c=='$') return(c);
else printf("erreur lexicale");
}
int yyerror(char *s){
printf("%s \n",s);
return 0;
}
int main(){
yyparse();
}
```

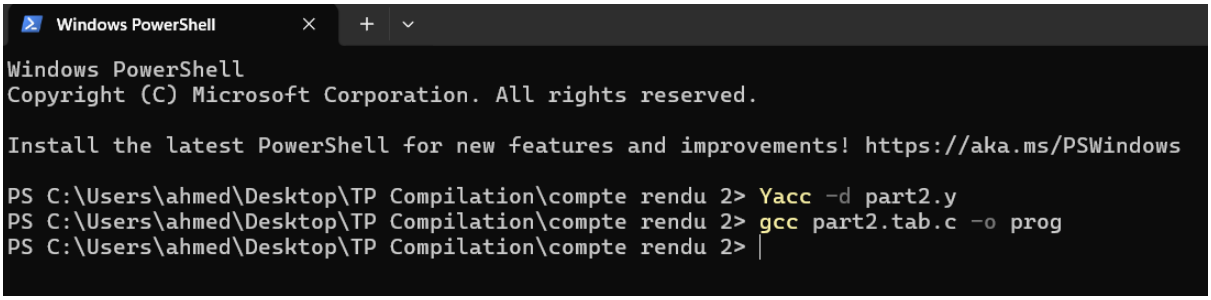
2)



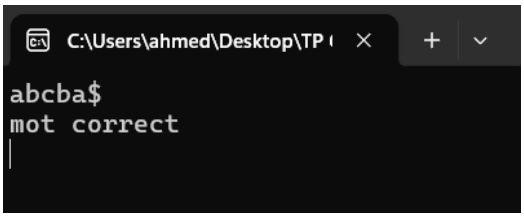
3)

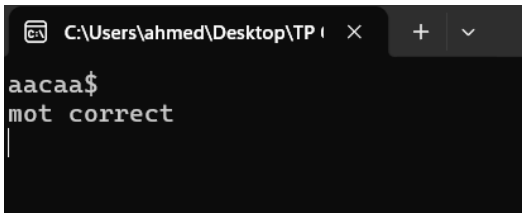


4)



5)





Each word contains exactly one central **c**, is symmetrically surrounded by matching **a** or **b** characters, and ends with **$**.

Ex1:

1a)

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
int yyerror(char *);
%}
%%
mot : S '$' { printf("correct word\n"); };

S : 'a' S 'b'
  |
  ;

%%

int yylex(){
    char c = getchar();
    if (c == EOF)
        return 0;
    while (c == '\n' || c == ' '){
        c = getchar();
        if (c == EOF)
            return 0;
    }
    if (c == 'a' || c == 'b' || c == '$')
        return c;
    return c;
}

int yyerror(char *s){
    int c;
    printf("incorrect word\n");
    while ((c = getchar()) != '\n' && c != EOF);
    return 0;
}

int main(){
    while (!feof(stdin)){
        yyparse();
    }
    return 0;
}
```

1b)

```
%{
#include <stdio.h>

int yylex(void);
int yyerror(char *);
%}
%%
mot : S '$' { printf("correct word\n"); };

S : 'a' S 'b'
  | 'a' 'b'
  ;

%%
int yylex(){
    char c = getchar();
    if (c == EOF)
        return 0;
    if (c == '\n' || c == ' ')
        return yylex();
    if (c == 'a' || c == 'b' || c == '$')
        return c;
    return 0;
}

int yyerror(char *s){
    int c;
    printf("incorrect word\n");
    while ((c = getchar()) != '\n' && c != EOF);
    return 0;
}

int main(){
    while (!feof(stdin)){
        yyparse();
    }
    return 0;
}
```

2)

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
int yyerror(char *);
%}
%%
mot : S '$' { printf("correct word\n"); };

S : 'a' S 'b' 'b'
  |
  ;

%%

int yylex(){
    char c = getchar();
    if (c == EOF)
        return 0;
    while (c == ' ' || c == '\n'){
        c = getchar();
        if (c == EOF)
            return 0;
    }
    if (c == 'a' || c == 'b' || c == '$')
        return c;
    return c;
}

int yyerror(char *s){
    int c;
    printf("incorrect word\n");
    while ((c = getchar()) != '\n' && c != EOF);
    return 0;
}

int main(){
    while (!feof(stdin)){
        yyparse();
    }
    return 0;
}
```

3)

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
int yyerror(char *);
%}
%%
mot : S '$' { printf("correct word\n"); }
    ;

S : 'a' S 'c'
  | 'a' B 'c'
  ;

B : 'b' B
  |
  ;

%%

int yylex(){
    char c = getchar();
    if (c == EOF)
        return 0;
    while (c == ' ' || c == '\n'){
        c = getchar();
        if (c == EOF)
            return 0;
    }
    if (c == 'a' || c == 'b' || c == 'c' || c == '$')
        return c;
    return c;
}

int yyerror(char *s){
    int c;
    printf("incorrect word\n");
    while ((c = getchar()) != '\n' && c != EOF);
    return 0;
}

int main(){
    while (!feof(stdin)){
        yyparse();
    }
    return 0;
}
```

4)

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
int yyerror(char *);
%}
%%
mot : S '$' { printf("correct word\n"); };

S : 'a' S 'c'
  | 'b' S 'c'
  |
  ;

%%

int yylex(){
    char c = getchar();
    if (c == EOF)
        return 0;
    while (c == ' ' || c == '\n'){
        c = getchar();
        if (c == EOF)
            return 0;
    }
    if (c == 'a' || c == 'b' || c == 'c' || c == '$')
        return c;
    return c;
}

int yyerror(char *s){
    int c;
    printf("incorrect word\n");
    while ((c = getchar()) != '\n' && c != EOF);
    return 0;
}

int main(){
    while (!feof(stdin)){
        yyparse();
    }
    return 0;
}
```

5)

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
int yyerror(char *);
%}
%%
mot : S '$' { printf("correct word\n"); };

S : '(' S ')'
  | '(' ')'
  | S '(' S ')'
  ;

%%

int yylex(){
    char c = getchar();
    if (c == EOF)
        return 0;
    while (c == ' ' || c == '\n'){
        c = getchar();
        if (c == EOF)
            return 0;
    }
    if (c == '(' || c == ')' || c == '$')
        return c;
    return c;
}

int yyerror(char *s){
    int c;
    printf("incorrect word\n");
    while ((c = getchar()) != '\n' && c != EOF);
    return 0;
}

int main(){
    while (!feof(stdin)){
        yyparse();
    }
    return 0;
}
```

Part3:

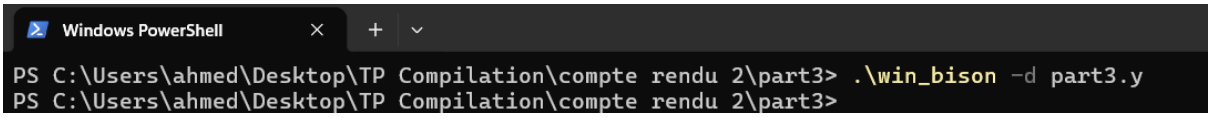
1)

part3.y

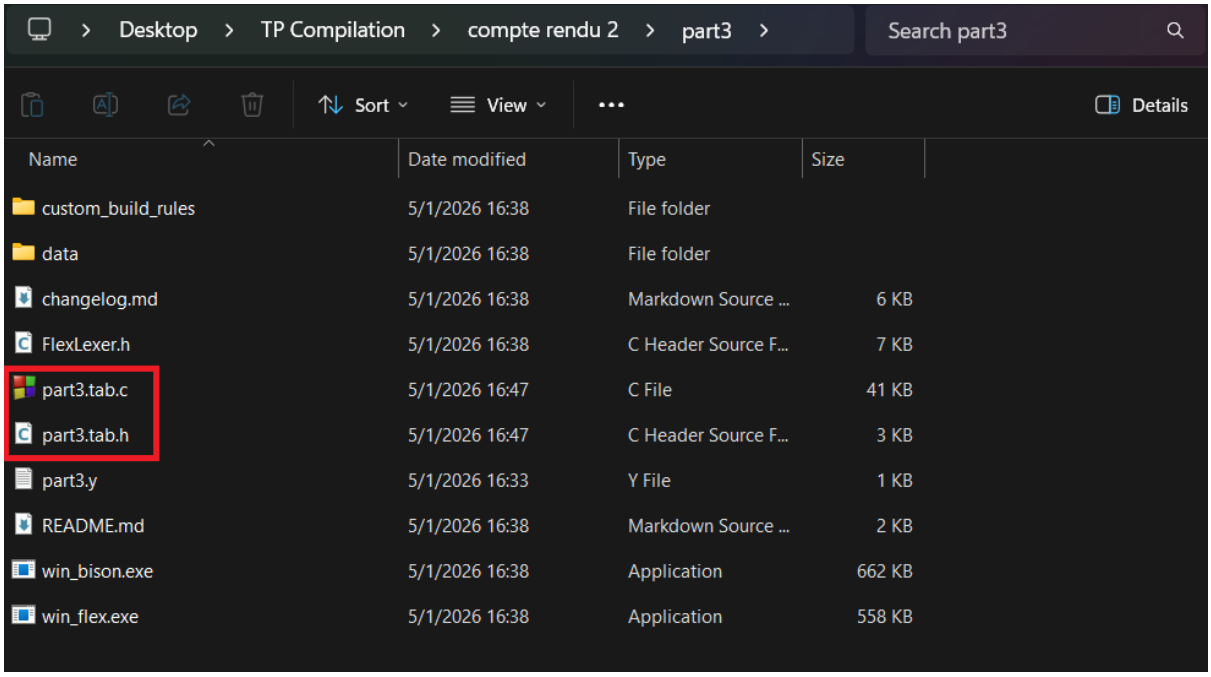
File Edit View

```
%{
#include<stdio.h>
int yylex(void);
int yyerror(char *s);
%}
%token FIN;
%token SOM;
%token PROD;
%token NB;
%%
liste:FIN {printf("correct");}
|SOM listesom'.'liste
|PROD listeprod'.'liste;
listesom: NB|listesom','NB;
listeprod: NB|listeprod','NB;
%%
#include "lex.yy.c"
int yyerror(char *s){printf ("%s", s);return (0);}
int main(){yyparse();getchar();}
```

2)



3)



4)

part3.l

File Edit View

```
%{
#include <stdio.h>
#include <math.h>
#include "part3.tab.h"
%}
```

5)

part3.l

File Edit View

```
%{
#include <stdio.h>
#include <math.h>
#include "part3.tab.h"
%}
%%
[0-9]+ {yylval=atoi(yytext); return NB;}
produit {return PROD;}
somme {return SOM;}
[,|.] {return yytext[0];}
[$] {return FIN;}
[ \t\n] {}
. {printf("Erreur");}
%%
int yywrap()
{return 1;}
```

6)

Windows PowerShell

```
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> .\win_flex part3.l
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3>
```

7)

part3.y

File Edit View

```
%{
#include <stdio.h>
int yylex(void);
int yyerror(char *s);
%}
%token FIN;
%token SOM;
%token PROD;
%token NB;
%%
liste:FIN {printf("correct");}
|SOM listesom'.'liste
|PROD listeprod'.'liste;
listesom: NB|listesom','NB;
listeprod: NB|listeprod','NB;
%%
#include "lex.yy.c"
int yyerror(char *s){printf ("%s", s);return (0);}
int main(){yyparse();getchar();}
```

8)

Windows PowerShell

```
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> gcc part3.tab.c -o prog
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3>
```

9)

Windows PowerShell

```
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> ./prog
somme 1, 2, 3. $
correct
somme 10, 20. produit 5, 2. $
syntax error
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3>
```

10)

part3.y

File Edit View

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex(void);
int yyerror(char *s);
%}

%token FIN SOM PROD NB
%start liste

%%
liste:
    FIN { return 0; }
    | SOM listesom '.' { printf("Somme = %d\n", $2); } liste
    | PROD listeprod '.' { printf("Produit = %d\n", $2); } liste
    ;

listesom:
    NB { $$ = $1; }
    | listesom ',' NB { $$ = $1 + $3; }
    ;

listeprod:
    NB { $$ = $1; }
    | listeprod ',' NB { $$ = $1 * $3; }
    ;

%%

#include "lex.yy.c"

int yyerror(char *s) {
    printf("Erreur: %s\n", s);
    return 0;
}

int main() {
    yyparse();
    return 0;
}
```

Windows PowerShell

```
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> .\win_bison -d part3.y
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> .\win_flex part3.l
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> gcc part3.tab.c -o prog
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> ./prog
somme 2,5. produit 3,6.$
Somme = 7
Produit = 18
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3>
```

11)

part3.l

File Edit View

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "part3.tab.h"
%}
%%
[0-9]+          { yylval = atoi(yytext); return NB; }
"somme"         { return SOM; }
"produit"       { return PROD; }
"soustraction"  { return SUB; }
"division"      { return DIV; }
","             { return ','; }
"."             { return '.'; }
"$"             { return FIN; }
[ \t\n]+        { ; }
.               { printf("Error: %s\n", yytext); }
%%
int yywrap() {
return 1;
}
```

part3.y

File Edit View

```
%%
liste:
    FIN { return 0; }
    | SOM listesom '.' { printf("Somme = %d\n", $2); } liste
    | PROD listeprod '.' { printf("Produit = %d\n", $2); } liste
    | SUB listesub '.' { printf("Soustraction = %d\n", $2); } liste
    | DIV listediv '.' { printf("Division = %d\n", $2); } liste
    ;

listesom:
    NB { $$ = $1; }
    | listesom ',' NB { $$ = $1 + $3; }
    ;

listeprod:
    NB { $$ = $1; }
    | listeprod ',' NB { $$ = $1 * $3; }
    ;

listesub:
    NB { $$ = $1; }
    | listesub ',' NB { $$ = $1 - $3; }
    ;

listediv:
    NB { $$ = $1; }
    | listediv ',' NB {
        if ($3 == 0) { yyerror("Error: Division by zero!"); $$ = $1; }
        else { $$ = $1 / $3; }
    }
    ;

%%
#include "lex.yy.c"

int yyerror(char *s) { printf("%s\n", s); return 0; }
int main() {
    yyparse();
    return 0;
}
```

Windows PowerShell

```
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> .\win_bison -d part3.y
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> .\win_flex part3.l
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> gcc part3.tab.c -o prog2
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> ./prog2
somme 2,5. produit 3,6. soustraction 10,2. division 20,2. $
Somme = 7
Produit = 18
Soustraction = 8
Division = 10
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3>
```

a)

LexA.l

File Edit View

```
%{
#include "LexA.tab.h"
#include <stdlib.h>
%}
%%
[0-9]+    { yylval = atoi(yytext); return NUMBER; }
"+"       { return '+'; }
"-"       { return '-'; }
"*"       { return '*'; }
"/"       { return '/'; }
"("       { return '('; }
")"       { return ')'; }
"$"       { return '$'; }
[ \t\n]+ ;
.         { printf("Lexical Error\n"); }
%%
int yywrap(){
return 1;
}
```

LexA.y

File Edit View

```
%{
#include <stdio.h>
int yylex(void);
int yyerror(char *s);
%}
%token NUMBER
%%
input:

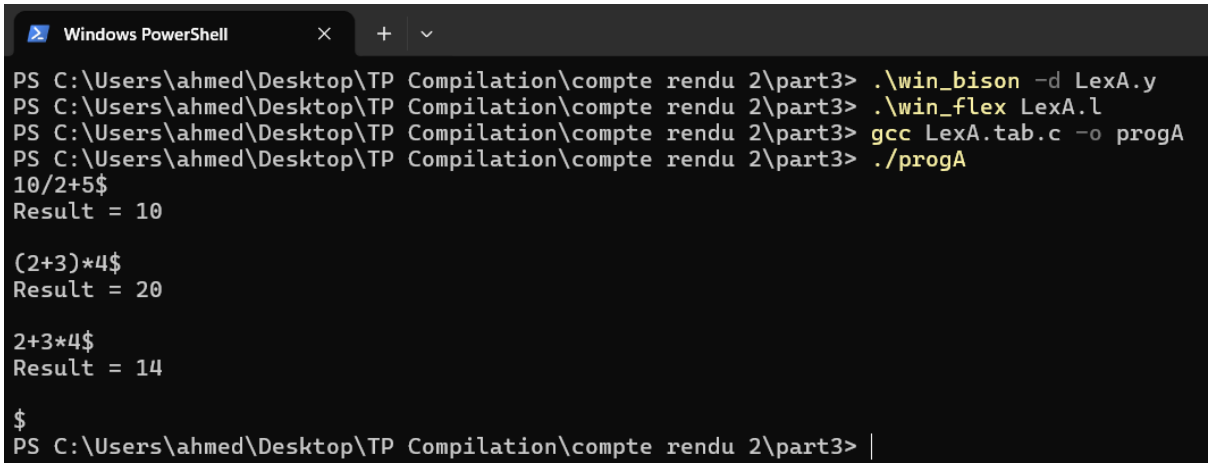
 | input line
 ;

line:
 '\n'
 | E '$' { printf("Result = %d\n\n", $1); }
 | '$'   { return 0; }
 ;

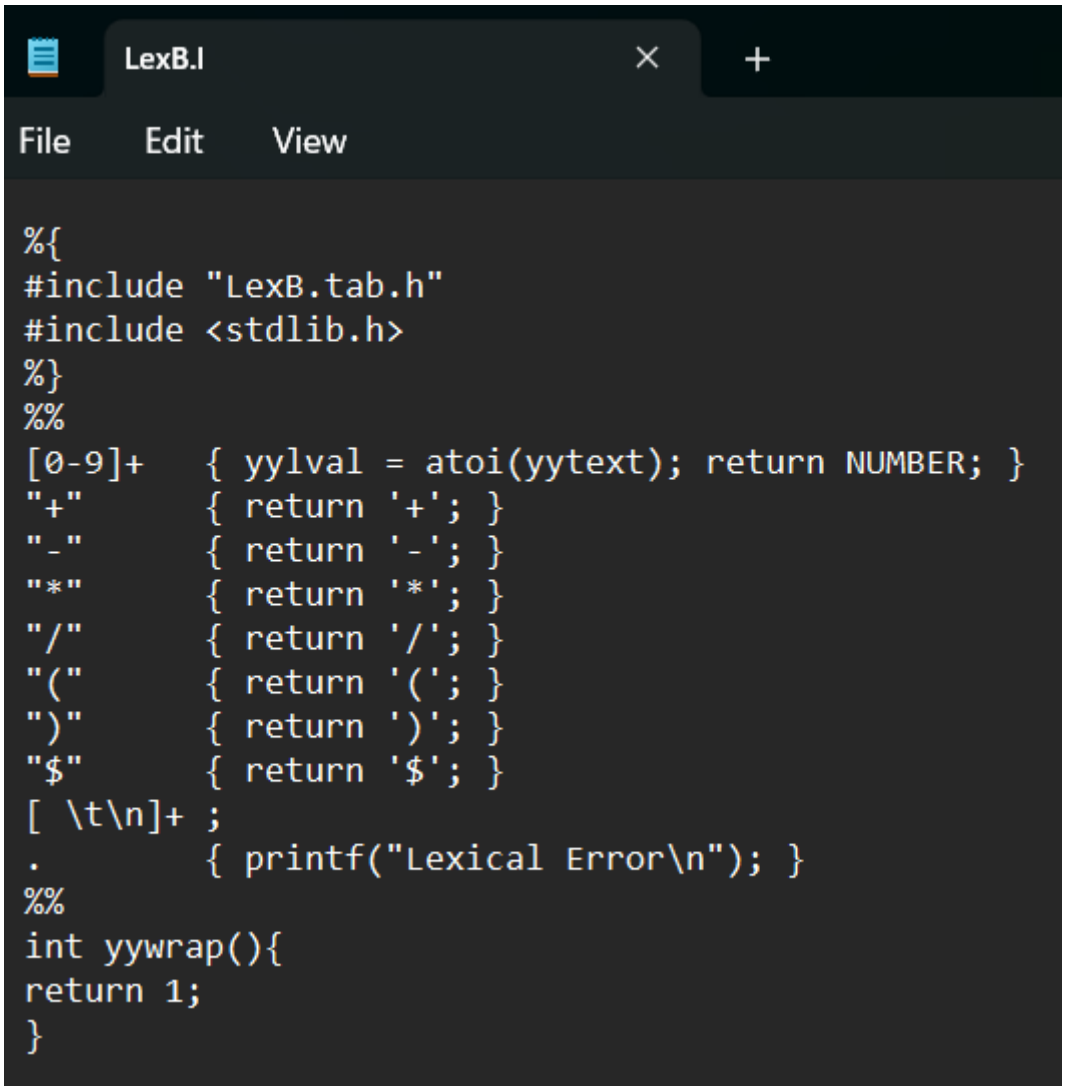
E: E '+' T { $$ = $1 + $3; }
 | E '-' T { $$ = $1 - $3; }
 | T { $$ = $1; } ;

T: T '*' F { $$ = $1 * $3; }
 | T '/' F { if($3 != 0) $$ = $1 / $3; else { yyerror("division by zero"); $$=0; } }
 | F { $$ = $1; } ;

F: NUMBER { $$ = $1; }
 | '(' E ')' { $$ = $2; } ;
%%
#include "lex.yy.c"
int yyerror(char *s){
printf("%s\n", s);
return 0;
}
int main(){
yyparse();
return 0;
}
```


Windows PowerShell
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> .\win_bison -d LexA.y
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> .\win_flex LexA.l
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> gcc LexA.tab.c -o progA
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> ./progA
10/2+5$
Result = 10
(2+3)*4$
Result = 20
2+3*4$
Result = 14
$
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3>

b)


LexB.l
File
Edit
View
%{
#include "LexB.tab.h"
#include <stdlib.h>
%}
%%
[0-9]+ { yylval = atoi(yytext); return NUMBER; }
"+" { return '+'; }
"-" { return '-'; }
"*" { return '*'; }
"/" { return '/'; }
"(" { return '('; }
")" { return ')'; }
"$" { return '$'; }
[\t\n]+ ;
. { printf("Lexical Error\n"); }
%%
int yywrap(){
return 1;
}

LexB.y

File Edit View

```
%{
#include <stdio.h>
int yylex(void);
int yyerror(char *s);
%}
%token NUMBER
%left '+' '-'
%left '*' '/'
%%
input:

  | input line
    ;

line:
      '\n'
    | E '$' { printf("Result = %d\n\n", $1); }
    | '$'   { return 0; }
    ;

E: E '+' E { $$ = $1 + $3; }
 | E '-' E { $$ = $1 - $3; }
 | E '*' E { $$ = $1 * $3; }
 | E '/' E { if($3 != 0) $$ = $1 / $3; else { yyerror("division by zero"); $$=0; } }
 | NUMBER { $$ = $1; }
 | '(' E ')' { $$ = $2; } ;
%%
#include "lex.yy.c"
int yyerror(char *s){
printf("%s\n", s);
return 0;
}
int main(){
yyparse();
return 0;
}
```

Windows PowerShell

```
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> .\win_bison -d LexB.y
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> .\win_flex LexB.l
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> gcc LexB.tab.c -o progB
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3> ./progB
9-4-3$
Result = 2

6+7*4$
Result = 34

40/(4*2)$
Result = 5

$
PS C:\Users\ahmed\Desktop\TP Compilation\compte rendu 2\part3>
```

c) **No.**

The two grammars do not describe the same language because they differ in structure and rules: the arithmetic grammar handles expressions with operators and parentheses, while the original grammar defines a sequence of numbers introduced by keywords like *somme* or *produit*, separated by commas and ending with specific symbols such as **.** or **$**

Furthermore, the original grammar lacks support for operator precedence and nested parentheses, which are core features of the arithmetic grammars (a) and (b).